OpenClaw & AgentCgroup

Open-Source AI Agents: Personal Assistants, Resource Characterization, and OS-Level Control

Yusheng Zheng (yzhen165@ucsc.edu)
PhD Student

Roadmap

Part I: OpenClaw

- What is OpenClaw?
- Architecture & Core Design
- Use Cases & Innovation
- Security Overview & CVEs

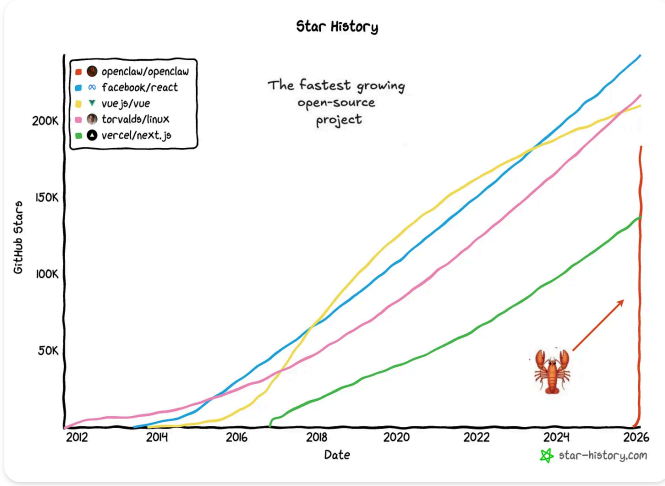
Part II: AgentCgroup

- Motivation: Why Study Agent Resources?
- Resource Characterization (144 tasks)
- Three Critical Mismatches
- eBPF Architecture & Bash Wrapper
- Bidirectional Resource Negotiation
- Evaluation & Takeaways

What is OpenClaw?

OpenClaw (formerly Clawdbot → Moltbot → OpenClaw) is a free, open-source, autonomous AI agent developed by Peter Steinberger.

- **MIT licensed**, fully open-source
- **Local-first**: memory stored as Markdown/YAML on your machine
- **Autonomous**: heartbeat daemon acts without prompting
- **Model-agnostic**: Claude, GPT, Gemini, Ollama, etc.
- **100k+ GitHub stars** in < 1 week (Jan 2026)
- Creator joined OpenAI; project moving to open-source foundation



OpenClaw: Use Cases & Innovation

Personal Assistant

- Calendar & task management
- Email drafting & replies
- Smart home control
- Cross-app orchestration

Developer Workflows

- Automated debugging & DevOps
- GitHub integration & PR management
- Cron-scheduled jobs & webhooks

Autonomous Tasks

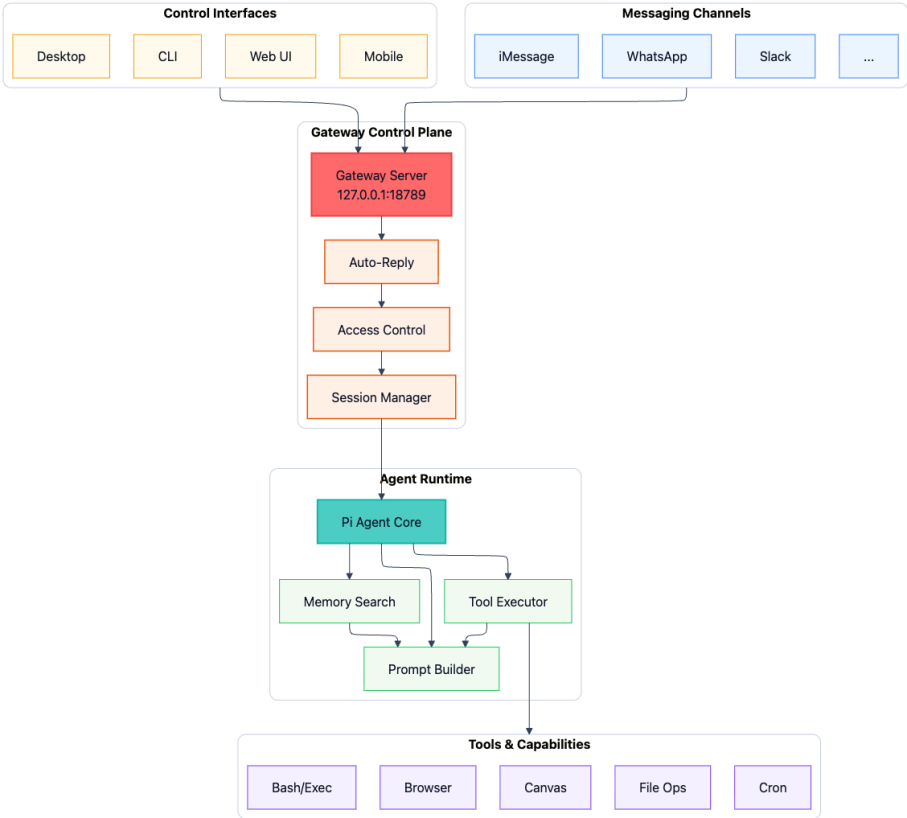
- Web scraping & form filling
- Browser automation
- Price negotiation (car buying!)

My Setup: 3 Agents, 3 Models

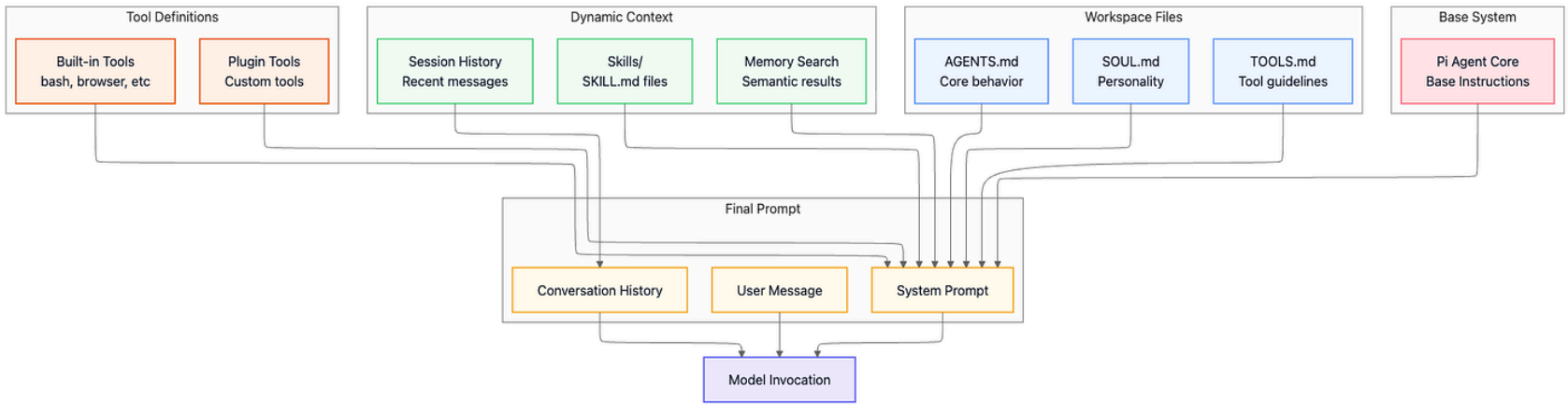
Initially used Claude, but account got banned. Switched to **Kimi, MiniMax, GLM** at **1/5~1/10 the cost**.

- Email handling
- Development tasks
- Open-source community maintenance
- Daily work
- Info search

OpenClaw: System Architecture



OpenClaw: Prompt Composition



What Makes OpenClaw Different

Heartbeat

Always-on background daemon; agent proactively monitors, responds, and acts without prompting

- continue do task without stop (unlike claude code, codex...)

Markdown = Memory & Persona

Personality, preferences, long-term memory all stored as Markdown; agent reads and edits them to evolve in the long run.

- local first, every memory is editable

Descriptive Skills, Not Code

Give permissions, describe the goal; agent builds its own tools and writes one-time code to complete tasks

- workflow is dead

Everything is CLI + Computer Use

Interactions through command line; naturally composable, scriptable; use Computer Use tools to do things like a human

- Orchestrate other agents (Gemini, Claude, Codex, etc.) via CLI
- Make tools with cli

Skills Ecosystem

ClawHub marketplace for modular, shareable skills; community-driven extensibility

- lots of risks

OpenClaw: Security Overview

The Threat Landscape

Self-hosted agents execute code with durable credentials and process untrusted input, creating dual supply chain risk

21,000+ → 42,665 exposed instances (Censys/Dayan); 5,194 actively vulnerable; 93.4% with auth bypass

ClawHub supply chain: 341 malicious skills (ClawHavoc campaign), grew to **824+** by Feb 16; ~20% of ecosystem compromised

Three Critical Dimensions

1. Identity: Gateway tokens = persistent credentials; token theft → full system access; no origin validation on WebSocket

2. Isolation: Skills & external instructions converge in same runtime; sandbox bypass trivial once auth compromised

3. Runtime Risk: Prompt injection = new SQL injection; agents execute arbitrary shell commands; governance lags adoption

OpenClaw: CVE Timeline (Jan–Feb 2026)

CVE / Advisory	Type	CVSS	Summary
CVE-2026-25253	RCE	8.8	1-Click auth token exfiltration via WebSocket hijacking
CVE-2026-25157	Command Injection	7.8	Unescaped SSH project path → arbitrary command execution
CVE-2026-26322	SSRF	7.6	Gateway tool endpoint allows internal network probing
CVE-2026-26319	Auth Bypass	7.5	Missing Telnyx webhook authentication
CVE-2026-26329	Path Traversal	High	Browser upload writes outside intended directory
GHSA-56f2	SSRF	7.6	Image tool endpoint allows internal requests
GHSA-pg2v	SSRF	6.5	Urbit auth endpoint allows internal requests
GHSA-c37p	Auth Bypass	6.5	Twilio webhook signature validation bypass

OpenClaw: Open Research Questions

- **How do we secure autonomous agents** that hold persistent personal credentials and execute arbitrary code? What's the threat model?
- **Is current OS resource management sufficient** for always-on agents running 24/7 via heartbeat daemons, and can create their own tools? Cgroups, schedulers, isolation…?
- **How do we build trust in skill ecosystems** when skills are descriptive, not code, and traditional code review doesn't apply? (341+ malicious skills)
- **What identity and access models** do we need for agents that act on behalf of users across multiple systems?
- **How do we balance autonomy and safety** as agents gain more capabilities (CLI, tool-building, computer use)? (People got their email deleted, password or personal info leaked by agent; but if not automatic, it's slow and inconvenient)

AgentCgroup: Overview

Problem

AI coding agents execute in **multi-tenant cloud sandboxes** but their OS-level resource behavior is **poorly understood**.

Key Findings

- **OS-level execution (tool calls + init) = 56~74% of end-to-end latency**; LLM reasoning only 26~44%
- **Memory, not CPU**, is the primary bottleneck for multi-tenant concurrency density
- Resource demands vary **20x across tasks** and **1.8x across runs** of the same task

AgentCgroup: Experimental Setup

144 SE tasks from SWE-rebench benchmark, with claude code

Config	Model	Tasks
Local GPU	GLM-4.7-Flash	111
Cloud API	Claude Haiku 4.5	33

Hardware: Intel Core Ultra 9 285K, 24 cores, 128GB DDR5

Environment: Podman containers, Linux 6.15, cgroup v2, 1s sampling

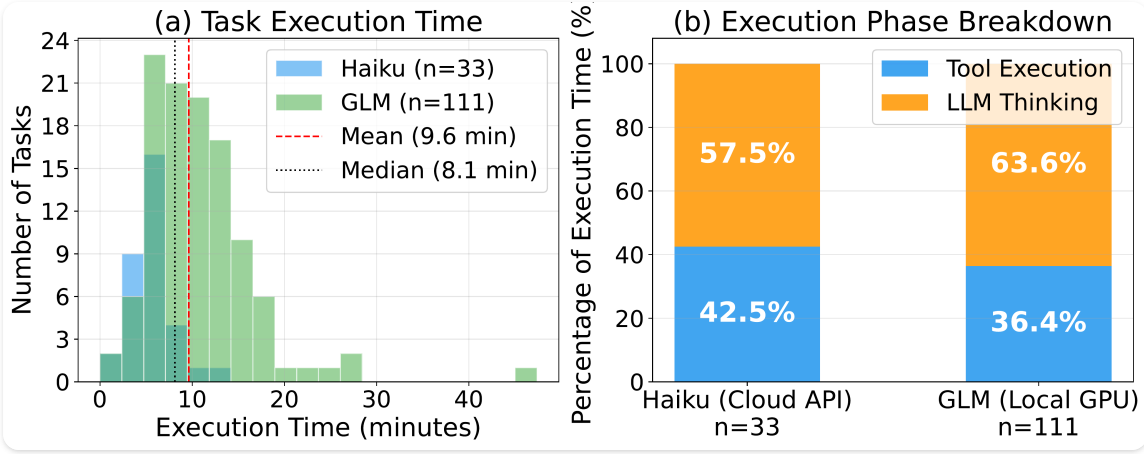
What We Measure

- **Execution model**: granularity at which resources vary
- **Temporal dynamics**: how fast controls must react
- **Cross-task variance**: whether demands can be predicted

Container Images

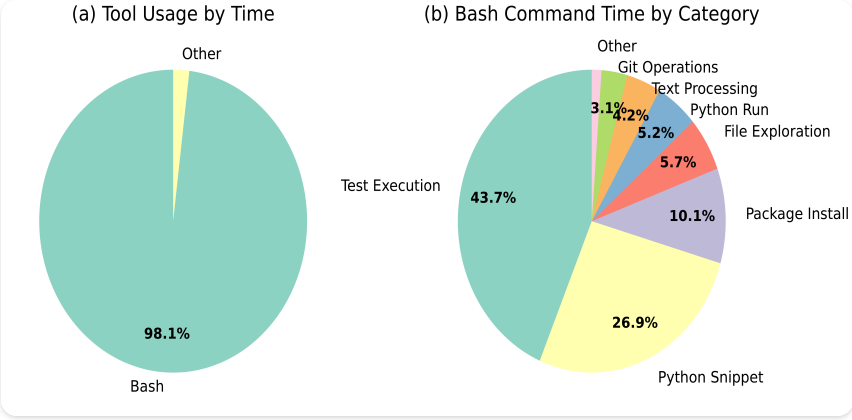
- **Average 3.5 GB** (range 2.9~17.3 GB)
- 7x larger than microservices, 70x larger than serverless

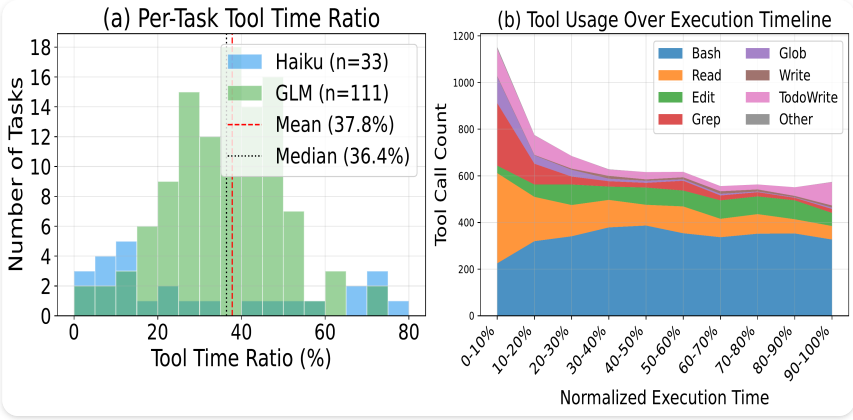
AgentCgroup: Execution Model



- **LLM reasoning accounts for 26~44% of end-to-end task latency; the remainder is consumed by tool execution (~40% of active time) and initialization (29~45%)**

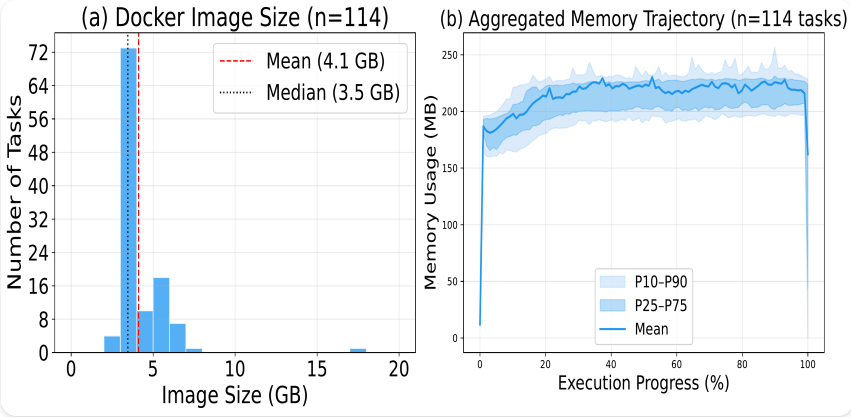
Tool Execution Breakdown

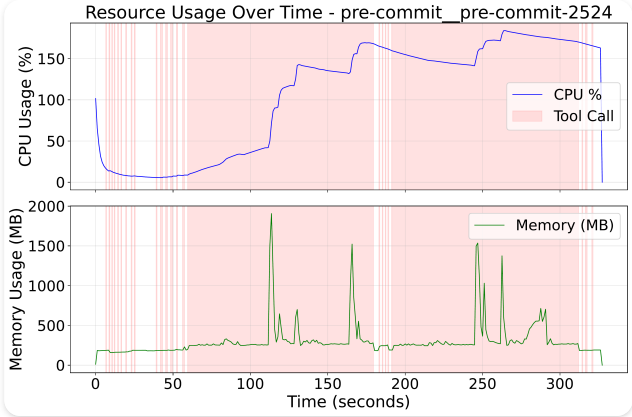




- **Bash dominates tool execution, spanning three orders of magnitude in duration** (98.1% of tool time in GLM)
- Tool calls follow a temporal "understand, modify, verify" pattern: Read concentrates in the first 30%, Bash peaks at 40~80%

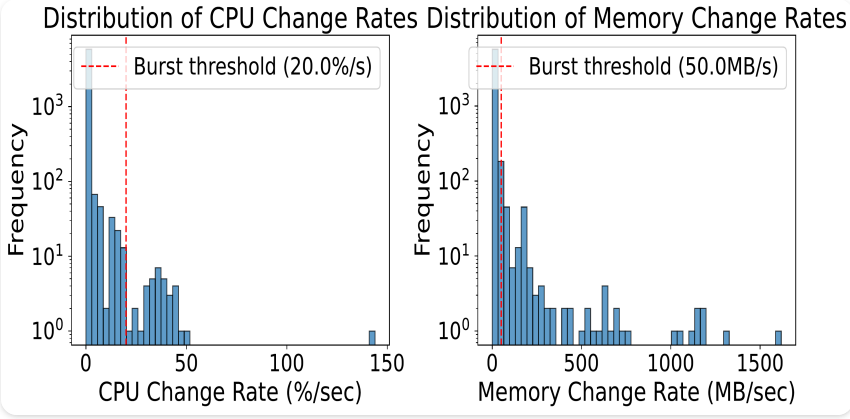
Resource Dynamics

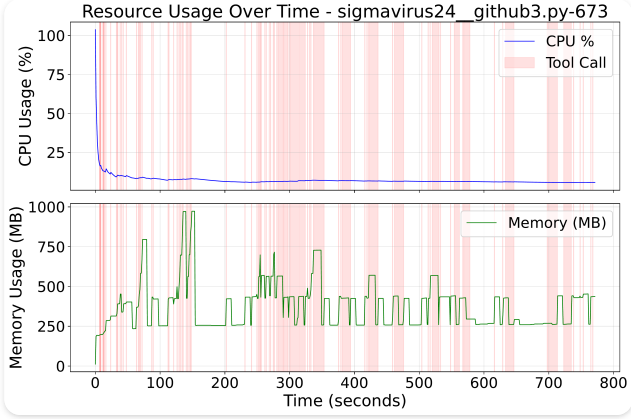




- **Resource consumption exhibits a two-layer structure: a ~185 MB framework baseline plus tool-call bursts**
- **Within the burst layer, resource consumption is determined by what the tool *does* (e.g., pytest vs. git status), not which tool is invoked (e.g., Bash vs. Read)**: Bash calls differ by 13.7x in peak memory depending on the command executed
- **Resource usage follows a burst-silence pattern, with 98.5% of memory bursts occurring during tool calls**

Change Rate & Cross-Model





- **Resource bursts last 1~2 seconds with peak-to-average ratio up to 15.4x**, several times beyond traditional cloud workloads
- **85%~97% of tasks contain retry loops with progressive memory accumulation**
- **CPU-memory correlation varies by task (−0.84 to +0.50); co-directional change cannot be assumed**

Table 1: Agent vs. Cloud Workloads

Dimension	Serverless/FaaS	Microservices	Batch/HPC	**AI Coding Agent**
Execution duration	100ms–2s	Long-running	Minutes–hours	**5–11 minutes**
Container image	~50 MB	100 MB–1 GB	1–10 GB	**2.9–17.3 GB (med. 3.5)**
Statefulness	Stateless	External state	Stateful	**In-process stateful**
Memory footprint	128–512 MB	Steady ~1 GB	Scales with data	**185 MB idle, peaks 2–4 GB**
Memory peak/avg	~1.5×	2–3×	~1×	**15.4×**
CPU utilization	Brief spike	10–40%	80–100%	**<13% avg, peaks >175%**
Determinism	Deterministic	Mostly deterministic	Deterministic	**1.8× variance same task**
Resource pattern	Flat	Steady + daily cycle	Stable rise	**Burst-silence alternating**
Termination cost	Just retry	Can migrate	Lose progress	**Lose all LLM context**

Table 2: Resource Management Mismatches

	Static Limits	Reactive Control	Predictive Scaling
Tools	mem.max/high, cpu.max; K8s QoS	PSI; oomd; TMO	VPA; Autopilot
Assumes	Known peak; stable demand	Gradual pressure; kill OK	Repeatable; history valid
Agent Reality	15.4× peak/avg; tool-semantic variation	1–2s burst; unpredictable	1.8× variance; kill = lose context
Mismatch	**Granularity**	**Responsiveness**	**Adaptability**

Granularity: Container-level policies cannot track tool-call-level dynamics. Setting `memory.max` at peak wastes >90%; at average triggers OOM kills.

Responsiveness: Memory bursts at 3 GB/s are too fast for user-space tools (systemd-oomd, K8s VPA). Need in-kernel μs-level enforcement.

Adaptability: 1.8× variance + non-determinism breaks history-based prediction. Kill-and-restart = triple penalty (cold start + lost context + non-deterministic re-execution).

AgentCgroup: Design & Evaluation

Three Mechanisms

1. Fine-Grained Resource Domains

- Hierarchical cgroup v2: per-tool-call ephemeral child cgroups
- Transparent bash wrapper creates `tool_<pid>_<ts>/`

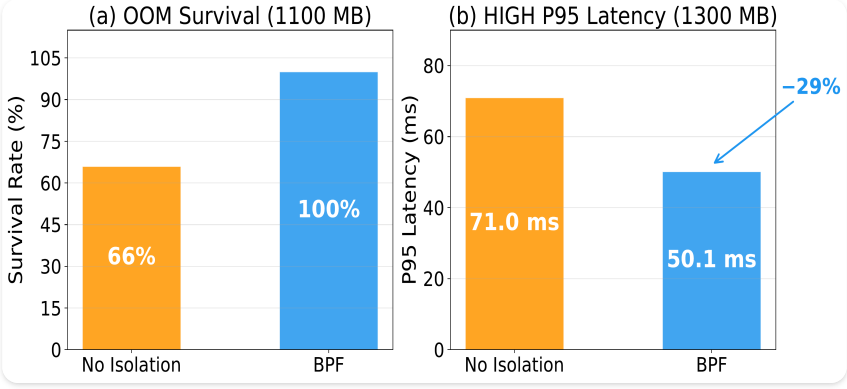
2. In-Kernel eBPF Enforcement

- `sched_ext` for CPU, `memcg_bpf_ops` for memory
- Graduated: **throttle → freeze → (never kill)**

3. Intent-Driven Coordination

- Agents declare: `AGENT_RESOURCE_HINT="memory:high"`
- Natural-language feedback on stderr when throttled

Evaluation Results



- **OOM survival**: 66% → **100%** (tight memory)
- **P95 latency**: **29% reduction** (moderate memory)
- **Wrapper overhead**: < 5ms per tool call
- **Hint accuracy**: 100% (Claude Haiku)

AgentCgroup: Next Steps

- Larger-scale evaluation with more models, tasks, and multi-tenant contention scenarios?
- openclaw?

Thank You

Questions?

References

OpenClaw GitHub · OpenClaw Blog · CVE-2026-25253 (NVD)

AgentCgroup Paper (arXiv:2602.09345) · AgentCgroup GitHub